Optimal Brain Surgeon: guía de estudio

Paper: B. Hassibi y D. G. Stork, *Second order derivatives for network pruning: Optimal Brain Surgeon*, NIPS 5 (1992/1993), pp. 164–171.

Contenido de esta guía: explicación completa del paper, las dudas que fuimos resolviendo (magnitud y OBD, Lagrange y saliencia, tamaño de la red, construcción recursiva de H^{-1}), ejemplos numéricos con figuras, el ejemplo XOR del paper y una guía práctica para aplicar OBS en un taller.

Nota sobre las figuras: las figuras 1 a 6 son ejemplos propios, construidos para explicar las ideas. Todos sus números se calcularon con el script `generar_figuras.py`. La figura 7 es un esquema redibujado de la Figura 3 del paper y muestra solo lo que el paper reporta.

Índice

1. La idea en una frase
2. El problema que resuelve el paper
3. Los métodos anteriores: magnitud y OBD
4. La derivación de OBS: Taylor, restricción y Lagrange
5. La saliencia: qué es y cómo se aplica
6. El algoritmo OBS
7. Cómo calcular H^{-1} de forma eficiente: la construcción recursiva
8. Por qué la aproximación $(t - o) \to 0$ es razonable
9. OBS y backpropagation: de dónde salen los gradientes
10. Resultados del paper: XOR, MONK y NETtalk
11. Ejemplo propio: poda iterativa completa
12. Conclusiones del paper
13. ¿Qué es una red "razonablemente grande"?
14. Limitaciones de OBS
15. Guía para el taller: cómo sacarle el máximo partido a OBS
16. Glosario
17. Referencias

---

1. La idea en una frase

Tienes una red ya entrenada con demasiados pesos y quieres quitar algunos. OBS usa la curvatura del error (la matriz Hessiana **completa**) para decidir **qué peso quitar**, y además **reajusta todos los demás pesos** para compensar la pérdida, sin reentrenar.

---

2. El problema que resuelve el paper

Una red con demasiados pesos sobreajusta: memoriza el ruido del conjunto de entrenamiento y generaliza mal. Una red con muy pocos pesos no consigue aprender. La estrategia del paper es empezar con una red grande, entrenarla y después podarla.

El paper plantea tres preguntas:

1. ¿Qué pesos hay que eliminar?
2. ¿Cómo hay que ajustar los pesos que quedan?
3. ¿Cómo se hace esto de forma computacionalmente eficiente?

Podar sirve para mejorar la generalización, reducir memoria y hardware, acelerar el entrenamiento posterior y, a veces, extraer reglas lógicas de la red.

---

3. Los métodos anteriores: magnitud y OBD

Los dos métodos anteriores podan de la misma forma. Calculan un **puntaje** para cada peso, ponen a cero el peso de menor puntaje y **no tocan los demás pesos**. Después normalmente se reentrena. Lo único que cambia entre ellos es el puntaje.

3.1 Poda por magnitud

Puntaje = |w_q|. Se elimina el peso más cercano a cero.

Ejemplo: con w = (0.05, −1.2, 0.3, 2.0, −0.08), los valores absolutos son 0.05, 1.2, 0.3, 2.0 y 0.08. El orden de eliminación es w_1, w_5, w_3, …

La idea es intuitiva: un peso pequeño debería aportar poco. El problema es que **no mira el error**. Un peso pequeño puede estar en una dirección donde el error es muy sensible, y entonces eliminarlo cuesta mucho.

3.2 Optimal Brain Damage (OBD, Le Cun, Denker y Solla, 1990)

Puntaje = ½·H_qq·w_q², donde $H_{qq} = \partial^2 E/\partial w_q^2$ es la curvatura del error en la dirección del peso q.

La fórmula sale de la expansión de Taylor. Si solo cambias w_q de su valor a 0, entonces δw = (0, …, −w_q, …, 0) y

$\delta E \approx \frac{1}{2}\cdot\delta w^{T}\cdot H\cdot\delta w = \frac{1}{2}\cdot H_{qq}\cdot w_q^2$

OBD estima, por tanto, **cuánto sube el error si eliminas el peso sin tocar nada más**. Le Cun et al. lo presentan suponiendo que H es diagonal, lo que facilita el cálculo.

Matiz importante: con la aproximación cuadrática, el puntaje de OBD es exacto para "quitar sin ajustar" aunque H no sea diagonal. Su defecto es que **ordena los pesos según ese número e ignora que los demás pesos podrían compensar**. OBS los ordena según el error que queda **después de la mejor compensación posible**.

3.3 Comparación con un mismo ejemplo

Usamos dos pesos con w* = (1.5, 1.0) y H = [[0.30, 0.35], [0.35, 1.50]].

Método	Puntaje $w_1 = 1.5$	Puntaje $w_2 = 1.0$	Elimina	Aumento real del error
Magnitud: |w|	1.5	**1.0**	w_2	**0.750**
OBD: ½·H_qq·w²	½·0.3·2.25 = **0.3375**	½·1.5·1 = 0.75	w_1	**0.3375**
OBS: w²/(2[H⁻¹]_qq)	**0.2456**	0.5458	w_1	**0.2456**

- **Magnitud** elimina w_2 porque es el más pequeño. Pero el error es muy sensible a w_2 (H_{22} = 1.5) y poco sensible a w_1 (H_{11} = 0.3).
- **OBD** detecta esa diferencia y elimina w_1, que es el peso grande.
- **OBS** elimina el mismo peso, pero además mueve w_2 de 1.0 a 1.35, y el error queda todavía más bajo.

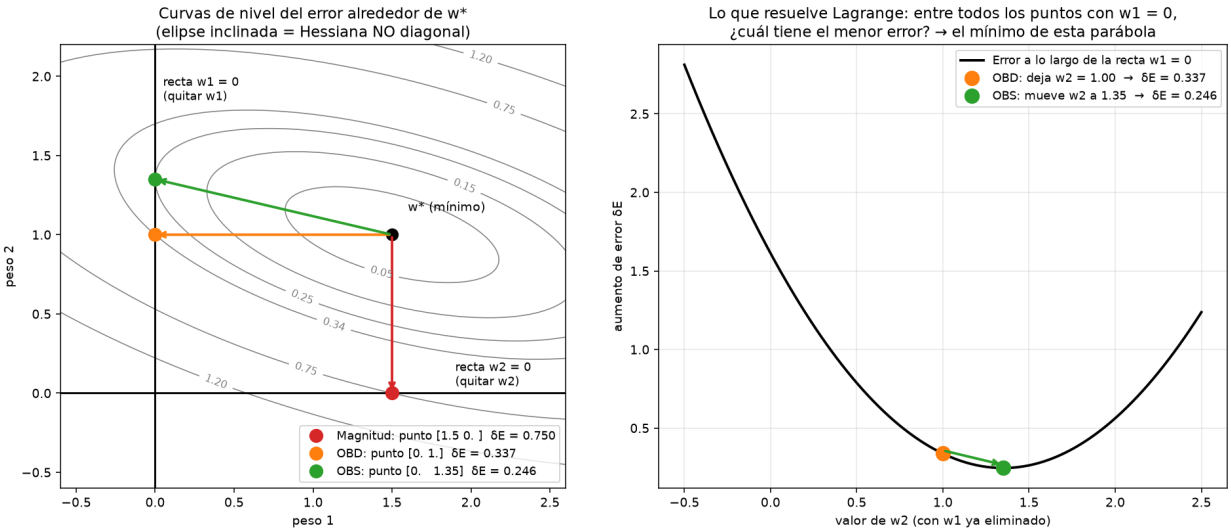


Figura 1. *Izquierda:* curvas de nivel del error alrededor del mínimo w*. Las elipses están inclinadas porque la Hessiana no es diagonal. Cada flecha muestra a dónde lleva cada método a la red. *Derecha:* el error a lo largo de la recta $w_1 = 0$, que contiene todas las redes posibles con w_1 eliminado. OBD se queda en $w_2 = 1.0$. OBS baja al fondo de la parábola, en $w_2 = 1.35$.

---

4. La derivación de OBS: Taylor, restricción y Lagrange

4.1 Aproximar el error con Taylor (Ec. 1)

Si los pesos cambian en δw, el error cambia en

$$\delta E = (\partial E/\partial w)^{\top}\cdot\delta w + \tfrac{1}{2}\cdot\delta w^{\top}\cdot H\cdot\delta w + O(\|\delta w\|^3)$$

- **$H = \partial^2 E/\partial w^2$** es la **Hessiana**, la matriz con todas las segundas derivadas $\partial^2 E/\partial w_i \partial w_j$. Describe la curvatura del error.
- La red está entrenada hasta un **mínimo local**, así que el gradiente es cero y el primer término desaparece.
- Los términos de orden 3 o superior se ignoran.

Queda: **$\delta E \approx \frac{1}{2} \cdot \delta w^T \cdot H \cdot \delta w$**

4.2 La restricción "eliminar el peso q" (Ec. 2)

Eliminar w_q significa que su nuevo valor tiene que ser 0:

$e_q^T \cdot \delta w + w_q = 0$

Aquí e_q es un vector de ceros con un 1 en la posición q. La ecuación dice que el cambio del peso q es exactamente −w_q.

4.3 El problema de optimización (Ec. 3)

La pregunta es: "*entre todas las formas de mover los pesos que dejan w_q = 0, ¿cuál sube menos el error?*". Y después: "*¿qué peso q conviene elegir?*"

Min_q { Min_δw (½·δwᵀ·H·δw) sujeto a $e_q^T \cdot \delta w + w_q = 0$ }

La parte derecha de la Figura 1 muestra la idea en 2D. Al obligar $w_1 = 0$ solo puedes moverte por la recta vertical, y sobre esa recta el error es una parábola. El mejor punto es el fondo de esa parábola. Lagrange es la herramienta que encuentra ese fondo en n dimensiones.

4.4 Resolución con multiplicadores de Lagrange (Ecs. 4 y 5)

1. **Lagrangiano:** $L = \frac{1}{2} \cdot \delta w^T \cdot H \cdot \delta w + \lambda \cdot (e_q^T \cdot \delta w + w_q)$
2. **Derivar respecto a δw e igualar a 0:** $H \cdot \delta w + \lambda \cdot e_q = 0$, de donde **$\delta w = -\lambda \cdot H^{-1} \cdot e_q$**. *Interpretación geométrica:* en el óptimo, la elipse de error toca la recta (o el hiperplano) de la restricción de forma tangente.
3. **Sustituir en la restricción:** $-\lambda \cdot [H^{-1}]_{qq} + w_q = 0$, de donde **$\lambda = w_q / [H^{-1}]_{qq}$**.
4. **Calcular el error resultante:** $\delta E = \frac{1}{2} \cdot \lambda^2 \cdot e_q^T \cdot H^{-1} \cdot H \cdot H^{-1} \cdot e_q = \frac{1}{2} \cdot \lambda^2 \cdot [H^{-1}]_{qq}$.

4.5 El resultado central (Ec. 5)

> **$\delta w = -(w_q / [H^{-1}]_{qq}) \cdot H^{-1} \cdot e_q$** (cómo se ajustan todos los pesos)
> **$L_q = w_q^2 / (2 \cdot [H^{-1}]_{qq})$** (la saliencia: cuánto sube el error al eliminar q)

- $[H^{-1}]_{qq}$ es el elemento diagonal q de la **inversa** de la Hessiana. En general **no es igual a 1/H_qq**.
- $H^{-1} \cdot e_q$ es la columna q de H^{-1}. Indica en qué dirección se mueven los demás pesos para compensar.

4.6 OBS incluye a los métodos anteriores como casos particulares

- **Si H es diagonal:** $[H^{-1}]_{qq} = 1/H_{qq}$, la saliencia queda $L_q = \frac{1}{2} \cdot H_{qq} \cdot w_q^2$, que es **la fórmula de OBD**, y δw solo cambia el peso q.
- **Si H ∝ I:** la saliencia es proporcional a w_q^2, que es **el criterio de magnitud**.
- **FARM** (Kung y Hu, 1991) supone activaciones lineales y solo actualiza los pesos oculta→salida, así que también es un caso restringido.

---

5. La saliencia: qué es y cómo se aplica

5.1 Definición

> **La saliencia L_q es el mínimo aumento del error posible al eliminar el peso q, dejando que todos los demás pesos se reajusten de forma óptima.** Es el "precio" de quitar ese peso.

5.2 Intuición: el papel de $[H^{-1}]_{qq}$

Se cumple que

$$\mathbf{1 / [H^{-1}]_{qq} = H_{qq} - h_q^{T} \cdot H_{resto}^{-1} \cdot h_q \leq H_{qq}}$$

donde h_q es la columna q de H sin el elemento q, y H_{resto} es H sin la fila ni la columna q. Este valor se llama complemento de Schur. Representa **la curvatura en la dirección q que queda después de que los demás pesos se reacomodan**. Por tanto:

- Si **$[H^{-1}]_{qq}$ es grande**, la curvatura efectiva es pequeña: el peso es **redundante**, otros pesos pueden hacer su trabajo, y quitarlo cuesta poco.
- Si **$[H^{-1}]_{qq}$ es pequeño**, el peso es **insustituible**.

OBD usa H_{qq}, la curvatura cuando los demás pesos se quedan quietos. OBS usa $1/[H^{-1}]_{qq}$, la curvatura cuando los demás pesos se mueven a su mejor posición. Toda la diferencia entre los dos métodos está ahí.

5.3 Ejemplo completo con 3 pesos

Datos: **$\mathbf{w = (1, 2, 1.5)}$** y **$\mathbf{H = [[4, 1.5, 0.5], [1.5, 1, 0.5], [0.5, 0.5, 1]]}$**.

Paso A: invertir H. $H^{-1} = [[0.6, -1, 0.2], [-1, 3, -1], [0.2, -1, 1.4]]$, así que $\text{diag}(H^{-1}) = (0.6, 3, 1.4)$.

Paso B: calcular la saliencia de cada peso.

- $L_1 = 1^2 / (2 \cdot 0.6) =$ **0.833**
- $L_2 = 2^2 / (2 \cdot 3) =$ **0.667**, la menor
- $L_3 = 1.5^2 / (2 \cdot 1.4) =$ **0.804**

Paso C: elegir el peso de menor saliencia, w_2. Es el peso **más grande** y aun así es el más barato de quitar, porque $[H^{-1}]_{22} = 3$ es grande. Su curvatura efectiva es 1/3, frente a $H_{22} = 1$.

Paso D: ajustar todos los pesos con la Ec. 5. $\delta w = -(w_2 / [H^{-1}]_{22}) \cdot$(columna 2 de H^{-1}) $= -(2/3) \cdot (-1, 3, -1) = (+0.667, -2, +0.667)$ Los nuevos pesos son **(1.667, 0, 2.167)**. w_2 queda exactamente en 0, y w_1 y w_3 crecen para cubrir su función.

Paso E: comparar el error real de cada método.

Método	Elimina	Aumento del error
Magnitud	w_1 (el más pequeño)	2.000
OBD	w_3	1.125
OBS	**w_2** (el más grande)	**0.667**

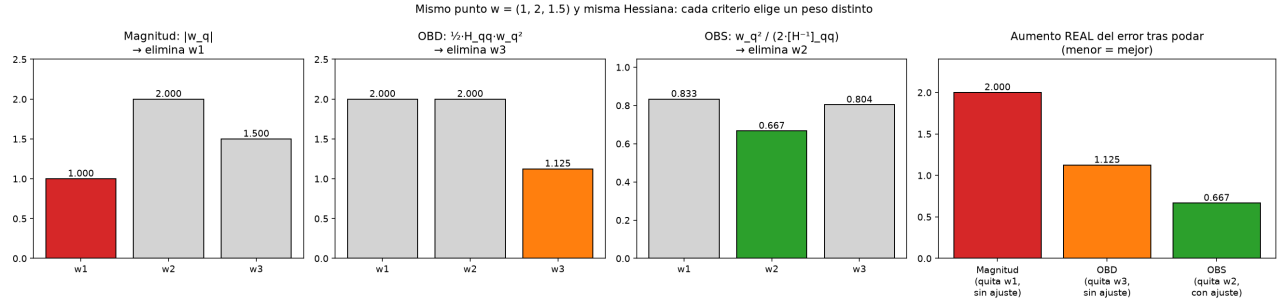


Figura 2. El mismo punto y la misma Hessiana, evaluados con los tres criterios. Cada criterio elige un peso distinto. El último panel muestra el aumento real del error: OBS deja tres veces menos error que magnitud.

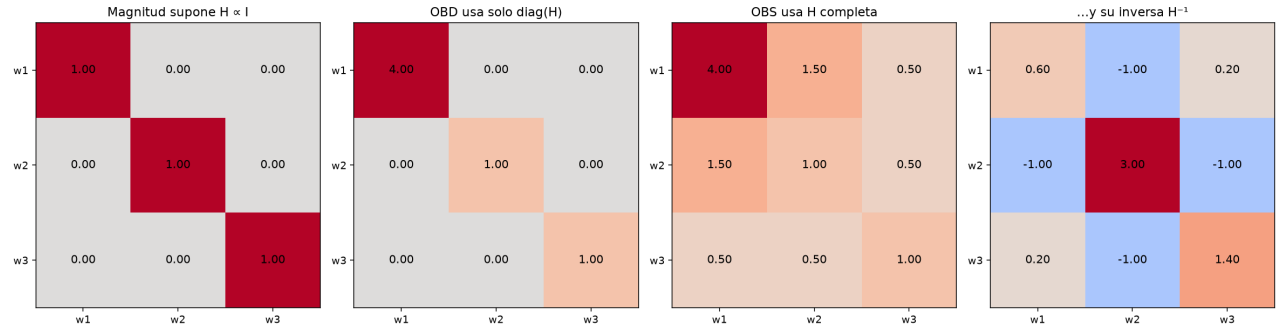


Figura 3. La información que usa cada método. Magnitud usa la identidad, OBD solo la diagonal de H, y OBS la H completa y su inversa. Los términos fuera de la diagonal, que OBD ignora, son los que permiten la compensación.

Curiosidad: con solo 2 pesos, OBS y OBD siempre eligen el mismo peso (como en la Figura 1 y en la Figura 1 del paper), aunque OBS deja menos error gracias al ajuste. Las diferencias en la elección aparecen a partir de 3 pesos.

---

6. El algoritmo OBS

1. Entrenar una red "razonablemente grande" hasta el error mínimo.
2. Calcular H^{-1}.
3. Buscar el q con menor saliencia $L_q = w_q^2 / (2[H^{-1}]_{qq})$. Si ese aumento de error es mucho menor que E, eliminar el peso q e ir al paso 4. Si no, ir al paso 5. (Se pueden usar otros criterios de parada.)
4. Usar ese q para actualizar **todos** los pesos con la Ec. 5. Volver al paso 2.
5. No se pueden eliminar más pesos sin un aumento grande de E. En este punto puede convenir reentrenar.

Para el error tras podar (antes de cualquier reentrenamiento), a segundo orden se cumple:

$E(\text{magnitud}) \geq E(\text{OBD}) \geq E(\text{OBS})$

Los autores llaman al método *Surgeon* (cirujano) porque, además de eliminar pesos, **recalcula y cambia** los demás sin descenso de gradiente ni reentrenamiento incremental.

---

7. Cómo calcular H^{-1} de forma eficiente: la construcción recursiva

El paso 2 parece imposible: invertir una matriz de n×n con miles o millones de pesos. La Sección 3 del paper lo resuelve.

7.1 La Hessiana del error cuadrático medio (Ecs. 6 a 9)

La red es o = F(w, in). El error es

$$E = (1/2P)\cdot\Sigma_k\ (t^{(k)} - o^{(k)})^T(t^{(k)} - o^{(k)})$$

donde P es el número de patrones, $t^{(k)}$ la salida deseada y $o^{(k)}$ la salida de la red. Su Hessiana exacta es

$$H = (1/P)\cdot\Sigma_k\ [\ (\partial F/\partial w)\cdot(\partial F/\partial w)^T - (\partial^2 F/\partial w^2)\cdot(t^{(k)} - o^{(k)})\ ]$$

El vector w puede representar cualquier parámetro continuo: pesos, parámetros de la función de transferencia, pesos compartidos, etc. Tampoco importa cómo se entrenó la red (backprop, Boltzmann, aprendizaje competitivo…), siempre que se puedan tomar derivadas.

7.2 La aproximación clave (Ecs. 10 a 14)

En un mínimo bien entrenado, o ≈ t, así que se elimina el segundo término:

> **$H \approx (1/P)\cdot\Sigma_k\ X^{(k)}\cdot X^{(k)T}$**, con **$X^{(k)} = \partial F(w, in^{(k)})/\partial w$**

- $X^{(k)}$ es el gradiente de la **salida de la red** (no del error) respecto a los pesos, para el patrón k.
- **H es la matriz de covarianza muestral de esos gradientes.** Hoy esta aproximación se conoce como Gauss-Newton o Fisher empírica.
- Con varias salidas, $X^{(k)}$ es una matriz n×n_o y se suma también sobre las salidas: $H = (1/P)\cdot\Sigma_k\ \Sigma_l\ X_l^{(k)}\cdot X_l^{(k)T}$ (Ec. 14).

7.3 La idea de la recursión

H es una **suma de P piezas sencillas**, una por patrón. Cada pieza $X\cdot X^T$ es una matriz de **rango 1**. Por eso H se puede construir sumando patrón a patrón (Ec. 15):

$$H_{m+1} = H_m + (1/P)\cdot X^{(m+1)}\cdot X^{(m+1)T}, \text{ con } H_0 = \alpha I \text{ y } H_P = H$$

Hay un truco algebraico: si conoces A^{-1} y le sumas una matriz de rango 1, puedes obtener la nueva inversa **sin invertir nada**. Es la fórmula de inversión de matrices (Woodbury, Ec. 16):

$$(A + B\cdot C\cdot D)^{-1} = A^{-1} - A^{-1}\cdot B\cdot(C^{-1} + D\cdot A^{-1}\cdot B)^{-1}\cdot D\cdot A^{-1}$$

Con A = H_m, B = X, C = 1/P y D = X^T, el término $(C^{-1} + D\cdot A^{-1}\cdot B)$ es un **solo número**: $P + X^T\cdot H_m^{-1}\cdot X$. Así se obtiene la **Ec. 17**:

> **$H^{-1}_{m+1} = H^{-1}_m - (H^{-1}_m \cdot X \cdot X^T \cdot H^{-1}_m) / (P + X^T \cdot H^{-1}_m \cdot X)$**, con $H_0^{-1} = \alpha^{-1} \cdot I$

(El paper dice "applied to each term in the analogous sequence in Eq. 16". Es una errata y se refiere a la Ec. 15.)

7.4 El papel de α

- Al principio no se ha sumado ningún patrón y H = 0, que no se puede invertir. Por eso se empieza con **$H_0 = \alpha I$**, cuya inversa es trivial: I/α.
- Los autores usan **$10^{-8} \leq \alpha \leq 10^{-4}$** e indican que el método es poco sensible a su valor.
- En realidad la recursión calcula **$(H + \alpha I)^{-1}$**. Eso equivale a añadir una penalización $\alpha \|\delta w\|^2$ en el Lagrangiano. **Castiga los saltos grandes en el espacio de pesos** y ayuda a que la aproximación de Taylor siga siendo válida.

7.5 Ejemplo numérico paso a paso

Dos pesos, P = 3 patrones con X = (1,0), (1,1) y (0,1), y α = 0.01.

m = 0: H_0^{-1} = [[100, 0], [0, 100]]

m = 1, X = (1, 0):

- $H_0^{-1} \cdot X$ = (100, 0)
- $X^T \cdot H_0^{-1} \cdot X$ = 100, así que el denominador es 3 + 100 = 103
- Elemento [1,1]: $100 - 100^2/103$ = **2.91**
- **H_1^{-1} = [[2.91, 0], [0, 100]]**

El valor 100 de w_2 no cambia porque este patrón no da información sobre w_2.

m = 2, X = (1, 1):

- $H_1^{-1} \cdot X$ = (2.91, 100) y el denominador es 3 + 102.9 = 105.9
- [1,1]: $2.91 - 2.91^2/105.9$ = 2.83
- [2,2]: $100 - 100^2/105.9$ = 5.58
- [1,2]: $0 - 2.91 \cdot 100/105.9$ = **−2.75**
- **H_2^{-1} = [[2.83, −2.75], [−2.75, 5.58]]**

Aparece un término fuera de la diagonal porque este patrón mezcla los dos pesos.

m = 3, X = (0, 1):

- **H_3^{-1} = [[1.95, −0.96], [−0.96, 1.95]]**

Este resultado coincide exactamente con invertir H_3 directamente.

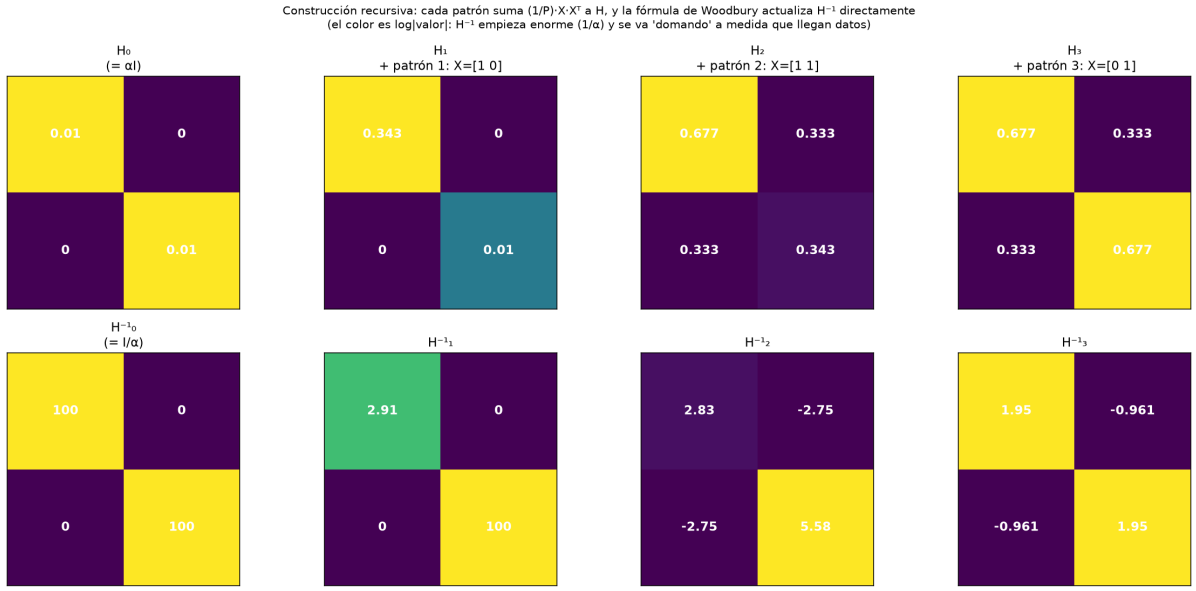


Figura 4. Fila superior: H se construye sumando un patrón cada vez. Fila inferior: H^{-1} se actualiza en cada paso con la Ec. 17, sin invertir ninguna matriz. El color representa log|valor|.

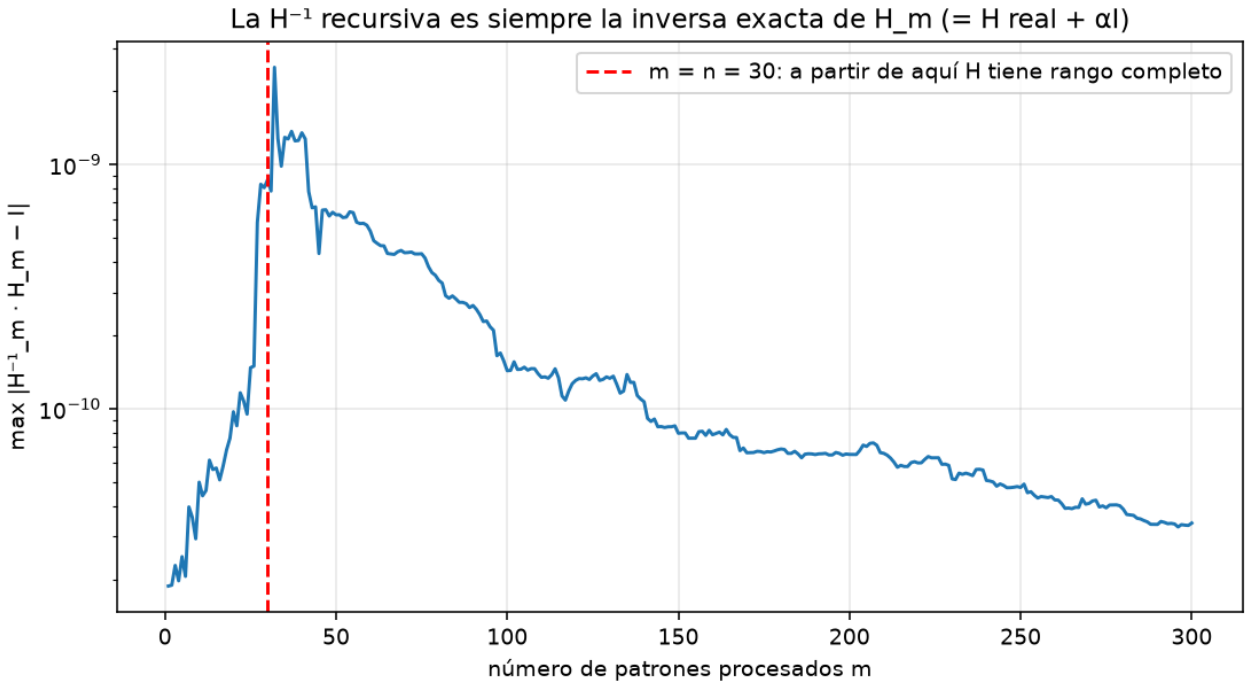


Figura 5. Verificación con 30 pesos y 300 patrones. La H^{-1} recursiva es siempre la inversa exacta de H_m; el error final es del orden de 10^{-11}.

7.6 Ventajas de la recursión

- **Una sola pasada por los datos:** cada patrón se usa una vez y se descarta. No hace falta guardar todos los gradientes.
- **Nunca se invierte una matriz grande:** solo hay productos matriz-vector y una división por un número.
- **Coste O(P·n²)**, el mismo que construir H.

8. Por qué la aproximación (t − o) → 0 es razonable (Sección 4)

Razones computacionales:

- La H verdadera suele ser **degenerada** (singular), sobre todo antes de podar mucho. La aproximación garantiza que H sea **definida positiva**, sin singularidades al invertirla.
- El coste de calcular H^{-1} es el mismo que el de calcular H: $O(P \cdot n^2)$.
- En estadística esta idea es la base del **método de scoring de Fisher**: sustituir la Hessiana por su valor esperado. Así se evitan los problemas de estabilidad de los métodos de Gauss-Newton.

Razones funcionales:

- En una red de alta capacidad entrenada hasta un error pequeño, el residuo (t − o) es básicamente **ruido**. La poda busca eliminar justo los pesos que aprenden ruido. Si se incluyera el residuo, cada paso de poda volvería a introducir ese ruido en el sistema.
- Otra forma de verlo: después de varias podas, la red está en un nuevo mínimo cuyo error puede no ser despreciable. Se imagina un objetivo efectivo **t*** que mantendría la red en ese mínimo. Lo que se pone a cero es (t* − o).

---

9. OBS y backpropagation: de dónde salen los gradientes (Sección 5)

Para una red de 3 capas con una salida (Figura 2 del paper), el vector $X^{(k)}$ tiene dos bloques (Ec. 19):

- **Pesos oculta→salida v_j (Ec. 20):** $X_v = f'(net) \cdot o_j$
- **Pesos entrada→oculta u_ji (Ec. 21):** $X_u = f'(net) \cdot f'(net_j) \cdot v_j \cdot o_i$

f es la función de activación, net la entrada neta de la neurona de salida, net_j la de la neurona oculta j, o_j la salida de la oculta j y o_i la de la entrada i. Son los mismos cálculos que hace backprop, así que **los gradientes que necesita OBS salen casi gratis**.

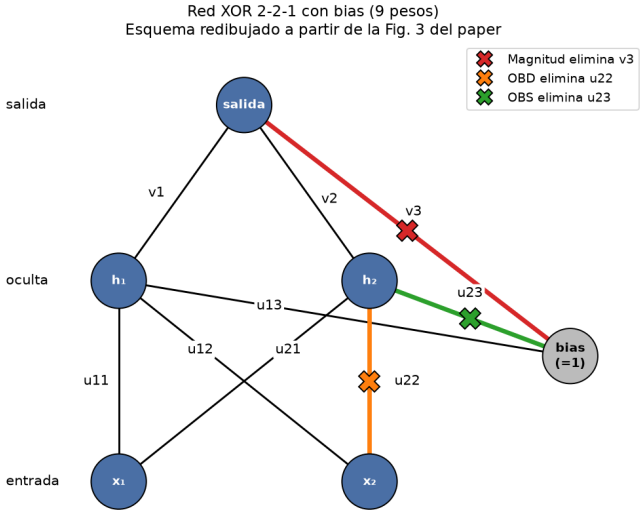
---

10. Resultados del paper (Sección 6)

10.1 El ejemplo XOR

El montaje:

- Una red **2-2-1 con unidad de bias**: 2 entradas, 2 neuronas ocultas y 1 salida, con **9 pesos** en total.
- Los pesos oculta→salida son **v_1, v_2 y v_3**, y los pesos entrada→oculta son **u_{11}, u_{12}, u_{13}, u_{21}, u_{22} y u_{23}**. Según la notación del paper (u_ji va de la entrada i a la oculta j) y su Figura 3, el índice 3 corresponde a la unidad de bias.
- La red se entrenó con los 4 patrones del XOR hasta un mínimo local con **error cero**. Después se usó cada método para **eliminar un solo peso**.



Resultado reportado en el paper (Sección 6)

Método	Elimina	¿Resuelve XOR sin reentrenar?	¿Resuelve XOR tras reentrenar?
Magnitud	v_3	No	No
OBD	u_{22}	No	No
OBS	u_{23} + ajusta los otros 8	Sí (error 0)	No hace falta

Figura 7. Esquema de la red XOR del paper (redibujado a partir de su Figura 3) y resultados reportados. En el original, el grosor de cada línea indica la magnitud del peso y los pesos inhibitorios (negativos) aparecen con línea discontinua.

Qué peso elimina cada método:

Método	Peso eliminado	Resultado
Magnitud	**v_3**	La red ya no resuelve XOR, **ni siquiera reentrenando**
OBD	**u_{22}**	La red ya no resuelve XOR, **ni siquiera reentrenando**
OBS	**u_{23}**	Ajusta los otros 8 pesos con la Ec. 5 y consigue **error cero sin reentrenar**

Las tres conclusiones del experimento:

1. **Cada método elimina un peso distinto.** Es el comportamiento de la Figura 2 de esta guía, pero en una red real.
2. **Elegir mal el peso es irreversible.** Con los pesos que eligen magnitud u OBD, el descenso de gradiente no consigue llevar el error de entrenamiento a cero. En palabras del paper, el error de esos métodos no se puede corregir entrenando más.
3. **OBS es consistente.** Los autores repitieron el entrenamiento desde varias condiciones iniciales, que llevan a mínimos locales distintos. En algunos casos OBD o magnitud acertaron, pero **solo OBS eliminó el peso correcto en todos los casos**.

La Hessiana de la red XOR (Figura 4 del paper):

- Se calculó con la Ec. 12 para la red entrenada y sin podar, y se muestra como un mapa de 9×9 con las filas y columnas etiquetadas $v_1 \ldots u_{23}$. El blanco indica valores grandes y el negro valores pequeños.
- Los pesos oculta→salida (v) tienen componentes grandes en la Hessiana, como era de esperar, porque la salida depende directamente de ellos.
- **La Hessiana está muy lejos de ser diagonal.** Los autores dicen haberlo encontrado, "para su sorpresa", en todos los problemas que estudiaron, incluidos los MONK. Esto explica por qué OBD, que supone H diagonal, se equivoca.

Cortes de la superficie de error (Figura 5 del paper):

- Son "rebanadas" 2D de la superficie de error de 9 dimensiones alrededor del mínimo w*.
- **Corte (v_3, u_{23}):** compara magnitud, que lleva v_3 a 0, con OBS, que lleva u_{23} a 0.
- **Corte (u_{22}, u_{23}):** compara OBD, que lleva u_{22} a 0, con OBS, que lleva u_{23} a 0.
- En cada corte se dibuja, sobre la superficie, la trayectoria desde w* que sigue cada método al eliminar su peso. El mensaje del pie de figura es que, para ese mínimo, **solo eliminar u_{23} permite que la red podada siga resolviendo XOR**. Tras actualizar los 9 pesos con la Ec. 5, el sistema queda con error cero (el paper lo indica, aunque no lo dibuja).

Lectura del ejemplo: la Figura 1 de esta guía muestra la misma idea a pequeña escala. Un criterio que mira solo el tamaño del peso o solo la diagonal de H puede dejar la red en una región de la que no se puede salir. OBS elige teniendo en cuenta cómo pueden compensar los demás pesos, y además hace esa compensación.

10.2 Problemas MONK

Son tres problemas de referencia basados en reglas lógicas. Se comparan con los resultados de Thrun et al. (1991), cuya red de backprop con weight decay (BPWD), ya regularizada, superó a todos los demás métodos en una competición amplia.

Problema	Precisión entrenamiento / test	Pesos BPWD	Pesos OBS	OBS usa	Reducción
MONK 1	100 / 100 (ambos)	58	14	24%	76%
MONK 2	100 / 100 (ambos)	39	15	38%	62%
MONK 3	93.4 / 97.2 (ambos)	39	4	10%	90%

- OBS consigue **la misma precisión** con muchos menos pesos y **sin reentrenar**.
- El aumento de error L de la Ec. 5 apenas afectó a la precisión.
- Las redes resultantes son tan simples que **se pueden recuperar las reglas lógicas** que generaron los datos, por ejemplo con el método de Towell y Shavlik (1992). Esto responde a la crítica de que las redes neuronales son ininteligibles.

(El resumen del paper cita las reducciones en otro orden: 90%, 76% y 62%.)

10.3 NETtalk (texto a voz)

- La red original de Sejnowski y Rosenberg (1987) tenía **18 000 pesos**.
- Los autores empezaron con **5546 pesos**. Tras entrenar con backprop, el error de test era **5259**.
- Podaron con OBS hasta **2438 pesos**, reentrenaron y volvieron a podar hasta **1560 pesos**, con un error de test de **4701**.
- Resultado: menos pesos y mejor generalización. Esto muestra que OBS se puede aplicar a problemas reales con miles de parámetros, como reconocimiento de voz o de caracteres.

---

11. Ejemplo propio: poda iterativa completa

Para ver el algoritmo completo (pasos 2 → 3 → 4 → repetir) en acción, se simuló un modelo con **12 pesos**, de los que solo **3 son útiles**. Las entradas están muy correlacionadas, que es lo que hace que H no sea diagonal. Se eliminan los pesos uno a uno con cada método y se mide el error de entrenamiento.

Pesos eliminados	**0**	**3**	**6**	**9 (quedan 3)**	**10**
Magnitud	0.042	0.055	0.157	0.169	0.601
OBD	0.042	0.055	0.124	0.169	0.601
OBS	0.042	**0.042**	**0.042**	**0.043**	**0.076**

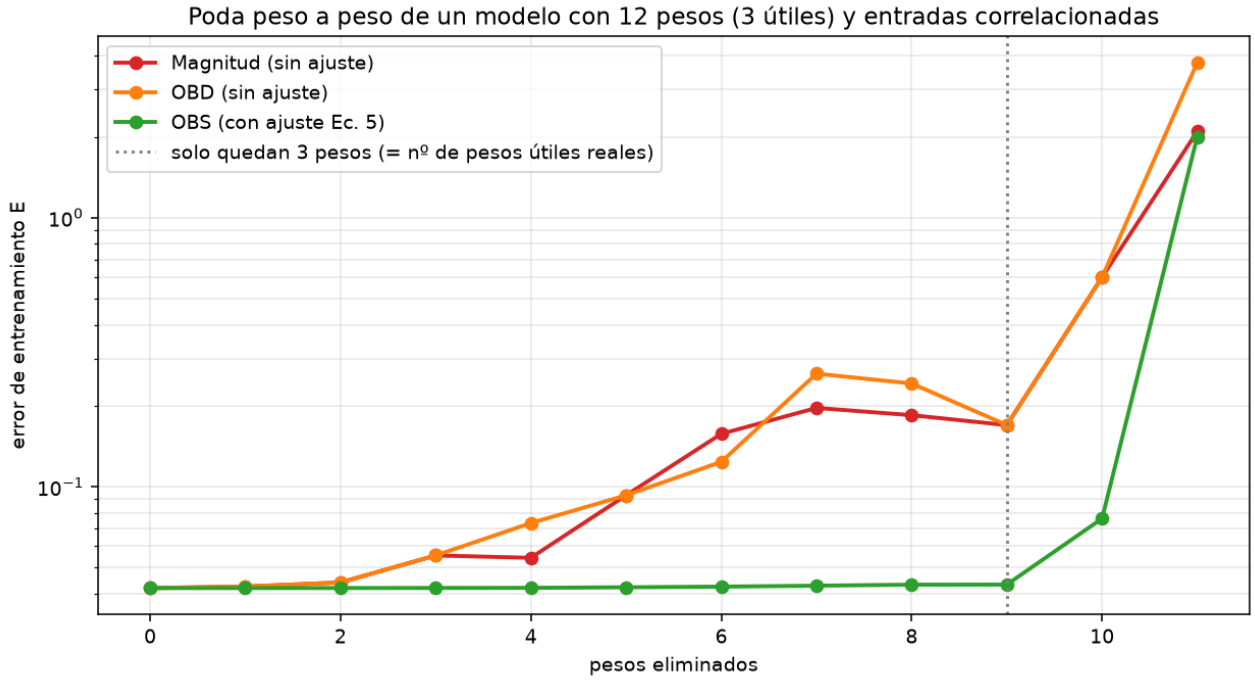


Figura 6. Error de entrenamiento (escala logarítmica) según el número de pesos eliminados.

- OBS elimina los 9 pesos sobrantes sin que el error suba, y sin reentrenar.
- Magnitud y OBD acumulan errores y terminan con unas cuatro veces más error con los mismos 3 pesos.
- Al pasar de 3 a 2 pesos todos empeoran, porque ya se está quitando un peso necesario. Ahí actúa el criterio de parada del paso 3: la saliencia se dispara.
- Las curvas de magnitud y OBD suben y bajan porque, sin ajuste, quitar un peso a veces compensa por casualidad un error anterior.

Advertencia: este modelo es lineal, así que la aproximación cuadrática es exacta y OBS equivale a reajustar los pesos restantes por mínimos cuadrados. En una red no lineal, OBS funciona igual pero de forma aproximada. Por eso, tras muchas podas, conviene reentrenar un poco, como hicieron los autores con NETtalk.

12. Conclusiones del paper (Sección 7)

- **Por qué falla la magnitud:** en la Figura 1 del paper, magnitud elimina el peso 2 y, al reentrenar, el peso 1 crece. La "solución" final es peso 1 grande y peso 2 = 0, justo **la opuesta** a la de OBS (peso 1 = 0, peso 2 grande). En una red grande, muchas decisiones equivocadas se acumulan. Según los autores, es "wishful thinking" esperar que el reentrenamiento lo arregle todo, sobre todo si la red ya está muy podada.
- **Por qué falla OBD:** en los problemas estudiados hay muchos términos fuera de la diagonal de H comparables a los de la diagonal.
- **OBS engloba a los métodos anteriores:** magnitud supone H ∝ I, OBD supone H diagonal y FARM supone activación lineal y solo toca los pesos de salida. Ninguna de esas suposiciones es válida ni suficiente para una eliminación óptima.
- **Extensiones que proponen los autores:**
- Restricciones distintas de w_q = 0, por ejemplo w_q = constante. Funcionan, pero esas redes tienen una longitud de descripción mayor.
- Eliminar **varios pesos a la vez**.
- Excluir los pesos de bias de la poda.
- Usar entropía cruzada o divergencia de Kullback-Leibler como error. En ese caso aparece la **matriz de información de Fisher** en lugar de la Hessiana.
- Combinar OBS con métodos graduales como el **weight decay** durante el entrenamiento.
- **OBS no define cuándo parar.** Se puede combinar con cualquier criterio de parada.

---

13. ¿Qué es una red "razonablemente grande"?

El paper no lo define con un número. Por el contexto, significa tres cosas:

1. **Que tenga más capacidad de la necesaria**, para llegar a un error de entrenamiento muy bajo, cerca del mínimo. OBS supone gradiente ≈ 0 y (t − o) ≈ 0. Si la red es tan pequeña que no aprende bien, esas suposiciones fallan y no hay pesos sobrantes que quitar.
2. **Que no sea tan grande que OBS se vuelva imposible de calcular.** H^{-1} tiene n^2 números. Con n = 10 000 pesos son 10^8 valores, unos 400 MB en float32. Con n = 10^6 pesos serían 4 TB.
3. **El tamaño que usan los propios autores:** 9 pesos en XOR (ya es el mínimo de la arquitectura 2-2-1), 58 y 39 en MONK (las redes de Thrun) y 5546 en NETtalk.

Regla práctica: una red que llegue a error de entrenamiento casi cero sin problemas, aunque sobreajuste un poco, y con un número de pesos tal que una matriz n×n quepa en memoria. Con redes modernas, OBS se aplica por capas o por bloques (sección 15.4), y la n relevante es la de cada bloque.

---

14. Limitaciones de OBS

- **Memoria $O(n^2)$:** era viable con miles de pesos, pero no con millones sin aproximaciones.
- **Recalcular H^{-1} en cada paso** (paso 2) es caro. Se puede evitar con la actualización de la sección 15.3, que el paper no detalla.

- **La aproximación de Taylor solo vale para cambios pequeños.** Si la saliencia predice poco error pero el ajuste δw es grande, la predicción puede fallar en una red no lineal.
- **Pocos datos frente a muchos pesos:** si P < n, la H de la Ec. 12 tiene rango ≤ P y es singular. Muchas saliencias quedan cerca de cero y las decisiones dependen sobre todo de α.
- **Poda no estructurada:** eliminar pesos sueltos reduce el almacenamiento, pero no siempre acelera la ejecución en hardware denso.

---

15. Guía para el taller: cómo sacarle el máximo partido a OBS

Esta sección reúne recomendaciones prácticas. Algunas vienen del propio paper y otras de trabajos posteriores que llevaron OBS a redes modernas. El archivo `obs_taller.py` contiene una implementación de referencia en numpy con las fórmulas de esta sección y una verificación numérica de cada una.

15.1 Preparación del experimento

1. **Define la línea base.** Entrena la red sin podar y guarda el error de entrenamiento, validación y test, el número de pesos y, si importa, la latencia y el tamaño en disco.
2. **Separa un conjunto de validación.** Úsalo para decidir cuándo parar de podar y para ajustar α. El test solo se usa al final.
3. **Entrena una red sobredimensionada hasta un buen mínimo** (sección 13). Si el gradiente no es casi cero, la Ec. 5 pierde precisión porque el término lineal de Taylor deja de ser despreciable.
4. **Combina con weight decay.** El paper indica que es compatible y que su punto de partida en MONK ya estaba regularizado.
5. **Compara siempre con magnitud y OBD**, con el mismo presupuesto de reentrenamiento, para medir qué aporta OBS.

15.2 Cálculo de la Hessiana

1. **Error cuadrático:** usa la Ec. 12 o la Ec. 14, con los gradientes de la salida por patrón (y por salida, si hay varias).
2. **Clasificación con entropía cruzada:** usa la **matriz de información de Fisher**, como sugieren los propios autores. En la práctica, la Fisher empírica se construye con los gradientes de la pérdida por muestra, $g^{(k)} = \nabla_w \ell^{(k)}$, y la misma recursión de la Ec. 17.
3. **Usa suficientes muestras.** Lo ideal es P ≥ n. Si P < n, H es singular y las saliencias dependen de α (sección 14). En redes grandes se trabaja por bloques para que esta condición se cumpla en cada bloque.
4. **Ajusta α en relación con la escala de H**, por ejemplo α = α_rel·media(diag(H)), y prueba varios valores de α_rel (10^{-4}, 10^{-3}, 10^{-2}) en validación. Un α mayor penaliza más los saltos grandes y hace OBS más conservador.

15.3 Eficiencia: no recalcules H^{-1} en cada paso

Tras eliminar el peso q y fijarlo en 0, la inversa del problema restringido se obtiene directamente:

> **$H^{-1}_nuevo = H^{-1} - (H^{-1} \cdot e_q \cdot e_q^{T} \cdot H^{-1}) / [H^{-1}]_qq$**

La fila y la columna q quedan en 0, y el resto coincide con invertir la submatriz de los pesos que quedan. Cada poda cuesta $O(n^2)$ en lugar de recalcular H^{-1} desde cero. Lo verifica `obs_taller.py`.

Esta actualización es exacta mientras te quedas en el mismo punto de linealización. Si después mueves mucho los pesos (por ejemplo, reentrenando), recalcula H^{-1} en el nuevo punto.

15.4 Escalar a redes grandes

- **Eliminar varios pesos a la vez (extensión del paper).** Para un conjunto Q de pesos: $\delta w = -H^{-1}[:, Q] \cdot (H^{-1}[Q, Q])^{-1} \cdot w_Q$ y $L_Q = \frac{1}{2} \cdot w_Q^{T} \cdot (H^{-1}[Q, Q])^{-1} \cdot w_Q$. Sirve también para **poda estructurada**: si Q son todos los pesos de una neurona, eliminas la neurona entera, y eso sí acelera la ejecución en hardware denso.
- **Aproximar H por bloques.** Ignorar la interacción entre capas o grupos y aplicar OBS dentro de cada bloque reduce la memoria de n^2 a la suma de los cuadrados de cada bloque. WoodFisher (Singh y Alistarh, 2020) estima la inversa de la Fisher empírica con la fórmula de Woodbury y una estructura por bloques, partiendo del marco OBD/OBS (arXiv:2004.14340).
- **OBS por capas sin reentrenamiento.** Optimal Brain Compression (Frantar, Singh y Alistarh, 2022) aplica OBS de forma exacta y eficiente capa por capa, usando un pequeño conjunto de datos de calibración, y lo extiende a cuantización (arXiv:2208.11580). SparseGPT (Frantar y Alistarh, 2023) lleva la idea a modelos tipo GPT y muestra que se pueden podar al menos al 50% en una sola pasada y sin reentrenar, con poca pérdida de precisión (arXiv:2301.00774). En estos métodos el objetivo de cada capa es $\|W \cdot X - \hat{W} \cdot X\|^2$, cuya Hessiana es proporcional a $X \cdot X^{T}$ y se comparte entre todas las filas de W.
- **Métodos sin matriz explícita.** M-FAC aproxima productos con la inversa de la Fisher sin guardar la matriz completa (arXiv:2107.03356).
- **Estima la memoria antes de empezar:** $n^2 \times 4$ bytes en float32. Si no cabe, usa bloques.

15.5 Respetar la validez de Taylor

1. **Poda de forma gradual:** por rondas (por ejemplo, entre un 5% y un 20% de los pesos restantes por ronda) en lugar de todo de golpe.
2. **Recalcula H en el nuevo punto** al empezar cada ronda.
3. **Reentrena brevemente entre rondas** cuando el error suba. Es la estrategia que siguieron los autores con NETtalk (podar, reentrenar, podar otra vez).
4. **Vigila el tamaño del ajuste $\|\delta w\|$.** Si es muy grande en relación con $\|w\|$, la predicción de la saliencia es poco fiable. Sube α o poda menos pesos por ronda.

15.6 Cuándo parar

OBS no trae un criterio de parada propio. Estas opciones se pueden combinar:

- **Umbral sobre la saliencia:** parar cuando L_q deje de ser "mucho menor que E", por ejemplo cuando supere una fracción fija de E (el paso 3 del algoritmo).
- **Saliencia acumulada:** parar cuando la suma de las saliencias eliminadas supere un presupuesto de error.

- **Validación:** dibujar el error de validación frente al número de pesos y elegir el punto donde empieza a subir con claridad.
- **Objetivo de recursos:** parar al llegar a un tamaño, una latencia o un porcentaje de dispersión fijados de antemano.

15.7 Detalles que mejoran el resultado

- **Protege los biases** si lo necesitas. El paper indica que se pueden excluir de la poda, y en `obs_taller.py` se hace con el argumento `protegidos`.
- **Reentrena al final**, con la máscara de pesos podados fija, para recuperar la precisión perdida por las aproximaciones.
- **Repite con varias semillas.** El paper muestra que el resultado depende del mínimo local, así que conviene reportar media y desviación.
- **Interpreta la red podada.** Con redes muy pequeñas, como en MONK, se pueden extraer reglas.

15.8 Esquema del procedimiento

```
1. Entrenar red sobredimensionada (con weight decay) hasta el mínimo
2. Repetir por rondas:
     a. Calcular gradientes por muestra (salida o pérdida) en el punto actual
     b. H⁻¹ = recursión de Woodbury con damping α                (Ec. 17)
     c. Repetir k veces dentro de la ronda:
          - L_q = w_q² / (2·[H⁻¹]_qq) para los pesos vivos y no protegidos
          - q* = argmin L_q
          - si L_q* supera el umbral → terminar
          - w ← w − (w_q*/[H⁻¹]_q*q*)·H⁻¹[:, q*]; w_q* = 0       (Ec. 5)
          - H⁻¹ ← H⁻¹ − H⁻¹[:, q*]·H⁻¹[q*, :] / [H⁻¹]_q*q*        (15.3)
     d. Medir el error de validación; si subió demasiado, reentrenar brevemente con la
máscara fija
3. Reentrenamiento final con máscara fija
4. Evaluar en test y comparar con magnitud y OBD
```

15.9 Qué medir y reportar

Métrica	Por qué
Número de pesos y % de dispersión	El objetivo principal de la poda
Error de entrenamiento, validación y test	Comprobar que la generalización se mantiene o mejora
Curva de error frente a pesos eliminados (como la Figura 6)	Comparar métodos de forma visual
Saliencia predicha frente a aumento real del error	Comprobar que la aproximación de Taylor se cumple
Tiempo de cálculo y memoria de H^{-1}	El coste real de OBS
Latencia y tamaño del modelo	El beneficio práctico (sobre todo con poda estructurada)
Media y desviación sobre varias semillas	Robustez de los resultados

15.10 Lista de comprobación

- [ ] Línea base entrenada y medida (entrenamiento, validación, test, tamaño)
- [ ] Gradiente cercano a cero al empezar a podar
- [ ] Hessiana adecuada a la pérdida (Gauss-Newton para MSE, Fisher para entropía cruzada)
- [ ] P ≥ n, o trabajo por bloques
- [ ] α ajustado en validación
- [ ] Actualización de H^{-1} tras cada poda, sin reinvertir
- [ ] Poda por rondas, con H recalculada y reentrenamiento breve entre rondas
- [ ] Criterio de parada definido de antemano
- [ ] Biases protegidos si conviene
- [ ] Comparación con magnitud y OBD con el mismo presupuesto
- [ ] Varias semillas

---

16. Glosario

Término	Significado
Poda (pruning)	Eliminar pesos de una red entrenada, poniéndolos a cero
Hessiana H	Matriz de segundas derivadas del error respecto a los pesos. Describe la curvatura del error
H no diagonal	Los pesos interactúan: cambiar uno modifica la sensibilidad del error a otro
Saliencia L_q	Mínimo aumento del error al eliminar el peso q con el mejor reajuste de los demás
$[H^{-1}]_{qq}$	Elemento diagonal q de la inversa de H. Cuanto mayor es, más redundante es el peso q
Multiplicador de Lagrange λ	Variable auxiliar para optimizar con restricciones
Fórmula de Woodbury / Sherman-Morrison	Permite actualizar una inversa tras sumar una matriz de rango bajo sin invertir de nuevo
α (damping)	Constante pequeña que hace H invertible y penaliza saltos grandes
Gauss-Newton / Fisher empírica	Aproximación $H \approx$ promedio de $X \cdot X^T$ que ignora el término del residuo
Información de Fisher	Equivalente a la Hessiana cuando la pérdida es entropía cruzada o KL
Complemento de Schur	$1/[H^{-1}]_{qq}$: curvatura en la dirección q después de reajustar los demás pesos
Poda estructurada	Eliminar neuronas, filtros o bloques enteros, no pesos sueltos

---

17. Referencias

- B. Hassibi y D. G. Stork (1993). *Second order derivatives for network pruning: Optimal Brain Surgeon.* Advances in Neural Information Processing Systems 5, 164–171.
- Y. Le Cun, J. S. Denker y S. A. Solla (1990). *Optimal Brain Damage.* NIPS 2, 598–605.
- S. Thrun et al. (1991). *The MONK's Problems: a performance comparison of different learning algorithms.* CMU-CS-91-197.
- T. J. Sejnowski y C. R. Rosenberg (1987). *Parallel networks that learn to pronounce English text.* Complex Systems 1, 145–168.
- S. P. Singh y D. Alistarh (2020). *WoodFisher: Efficient Second-Order Approximation for Neural Network Compression.* arXiv:2004.14340
- E. Frantar, S. P. Singh y D. Alistarh (2022). *Optimal Brain Compression: A Framework for Accurate Post-Training Quantization and Pruning.* arXiv:2208.11580
- E. Frantar y D. Alistarh (2023). *SparseGPT: Massive Language Models Can Be Accurately Pruned in One-Shot.* arXiv:2301.00774
- E. Frantar, E. Kurtic y D. Alistarh (2021). *M-FAC: Efficient Matrix-Free Approximations of Second-Order Information.* arXiv:2107.03356

Las descripciones de los trabajos posteriores están parafraseadas a partir de sus resúmenes en arXiv.